گزارش امن‌سازی پلتفرم سانس‌چی

مستند فارسیِ کارهای انجام‌شده در کد بک‌اند و فرانت — همراه با چک‌لیست استقرار و آزمون‌های خودکار

پروژه: reserve-football (سانس‌چی)
شاخهٔ کار: `security/hardening`
تاریخ گزارش: ۱۴۰۴/۰۷/۲۰ (۱۱ اکتبر ۲۰۲۵)
مخزن: backend (Express + MongoDB) و web (Next.js)

خلاصهٔ مدیریتی

هدف، بستن مسیرهای سوءاستفادهٔ رایج در یک اپ رزرو و پرداخت آنلاین بود: دسترسی غیرمجاز به API، دستکاری مبلغ/کمیسیون از سمت کلاینت، دور زدن OTP، جعل جایزهٔ بازی پنالتی، و پیکربندی ناامن در production. اصلاحات در لایهٔ HTTP (Helmet، CORS، rate limit)، احراز هویت (OTP امن، JWT)، منطق مالی (پرداخت زرین‌پال، تأیید اتمی)، و محدودیت فیلدهای قابل‌نوشتن (mass assignment) اعمال شد. یک اسکریپت دود (`npm run security:smoke`) برای بازبینی سریع ۹ سناریو اضافه شد.

فهرست

1. مشکلات و ریسک‌هایی که برطرف شدند
2. جزئیات فنی به تفکیک حوزه
3. فایل‌ها و ماژول‌های جدید/تغییریافته
4. متغیرهای محیط (env) مرتبط با امنیت
5. آزمون‌های خودکار (Security Smoke Test)
6. چک‌لیست استقرار در پنل میزبانی
7. کارهای باقی‌مانده و محدودیت‌ها
8. دستورات اجرا

۱. مشکلات و ریسک‌هایی که برطرف شدند

حوزه	ریسک قبلی (خلاصه)	اقدام انجام‌شده
مدیریت زمین (ادمین)	امکان ایجاد/ویرایش زمین با secret ساده یا بدون JWT؛ ارسال فیلدهای دلخواه در body	middleware `adminSecret`: اگر `ADMIN_SECRET` در header نباشد، **JWT + نقش admin** الزامی است. فقط فیلدهای لیست سفید با `pickPitchFields` ذخیره می‌شوند.
پنل مالک زمین	مالک می‌توانست فیلدهای حساس (مثل کمیسیون) را عوض کند یا در production رزرو را دستی «پرداخت‌شده» کند	PATCH مالک فقط فیلدهای مجاز (بدون `commissionAmount` و `owner`). وضعیت `paid` در production فقط اگر `ALLOW_OWNER_MANUAL_PAY=true`؛ پیش‌فرض **غیرفعال**. مسیر مجاز از `confirmReservationPayment`.
CORS و هدرهای HTTP	پذیرش مبدأهای نامشخص؛ افشای اطلاعات سرور	`CORS_ORIGINS` در production؛ Helmet؛ حذف `X-Powered-By`؛ `trust proxy` برای rate limit پشت Render؛ `X-Request-Id` برای ردیابی لاگ.
OTP ورود	کد ضعیف/قابل حدس؛ ذخیرهٔ plain text؛ spam SMS؛ افشای ثبت بودن شماره	مدل `OtpChallenge` در MongoDB؛ HMAC-SHA256 با pepper؛ کد ۶ رقمی؛ TTL و سقف تلاش؛ cooldown ارسال؛ پاسخ یکسان برای شمارهٔ ناموجود (anti-enumeration)؛ rate limit روی `/api/auth/`.
پرداخت زرین‌پال	اعتماد به مبلغ کلاینت؛ پرداخت رزرو دیگران	مبلغ و رزرو فقط از DB؛ فیلتر `user: req.user._id`؛ authority روی رزرو؛ verify سرور؛ تأیید idempotent و اتمی در `confirmReservationPayment`.
Mock pay / refund / wallet	فعال ماندن در production	`ALLOW_MOCK_PAY`، `ALLOW_MOCK_REFUND` در production باید false؛ topup کیف پول در production مسدود مگر mock صریحاً فعال باشد.
بازی پنالتی و کد تخفیف	ارسال مستقیم `goalsScored` بدون بازی واقعی	نشست `PenaltyGameSession`؛ `sessionId` یک‌بارمصرف؛ انقضا؛ حداقل زمان بازی در production (~۴۵ ثانیه)؛ rate limit جدا.
درخواست همکاری مالک	spam فرم عمومی	rate limit (۵ درخواست در ساعت به ازای IP)؛ اعتبارسنجی شبا (`lib/sheba.js`).
JWT	secret کوتاه یا پیش‌فرض در production	در `NODE_ENV=production` بدون `JWT_SECRET` ≥ ۳۲ کاراکتر، سرور خطا می‌دهد.

حوزه	ریسک قبلی (خلاصه)	اقدام انجام‌شده
Health / readiness	load balancer فکر می‌کند سرویس سالم است در حالی که DB قطع است	`api/health` → 503/ اگر MongoDB وصل نباشد؛ `api/health/live/` برای liveness.
فرانت Next.js	مسیرهای dev در production؛ OTP ۴ رقمی	middleware مسیرهای `payment/test/` و `mobile-preview/` را در production به خانه redirect می‌کند؛ ورود OTP ۶ رقمی؛ پیش‌نمایش UI با `uiPreview=1` query برای iframe (بدون دور زدن OTP واقعی در prod).
وابستگی‌ها	CVE در پکیج‌ها	افزودن `helmet`؛ ارتقای audit npm — `nodemailer` هنوز موارد high دارد (نیاز بررسی دوره‌ای).
مستندات استقرار	secretها در repo یا بدون راهنمای Render	`deploy-security-checklist-fa.md`، `render.example.yaml` (بدون secret واقعی)، به‌روزرسانی `backend/.env.example`.

۲. جزئیات فنی به تفکیک حوزه

۲.۱ لایهٔ HTTP (`backend/server.js`)

- **Helmet:** هدرهای امنیتی پیش‌فرض؛ `crossOriginResourcePolicy: cross-origin` برای assetها.
- **CORS:** فقط مبدأهای لیست `CORS_ORIGINS`؛ در development localhost مجاز؛ در production بدون env خالی = هیچ مبدأ مرورگری مجاز نیست.
- **Rate limiting:** `authLimiter` روی `api/auth/` (پیش‌فرض ۳۰ در ۱۵ دقیقه)؛ `apiLimiter` روی بقیهٔ API (۵۰۰ در ۱۵ دقیقه)؛ health و auth از apiLimiter مستثنی.
- **بدنهٔ JSON:** سقف اندازه با `JSON_BODY_LIMIT` (پیش‌فرض 4mb).
- **خطای سراسری:** handler یکپارچه؛ پیام عمومی در production؛ لاگ با `requestId`.
- **MongoDB:** خروج اجباری اگر `MONGO_URI` تنظیم نشده باشد.

۲.۲ OTP (`lib/otpStore.js`, `models/OtpChallenge.js`, `routes/auth.js`)

- تولید کد: `crypto.randomInt(100000, 999999)`.
- ذخیره: فقط hash با HMAC و pepper (`OTP_PEPPER` یا `JWT_SECRET`).
- مقایسه: `crypto.timingSafeEqual` برای جلوگیری از timing attack.
- محدودیت ارسال: ۶۰ cooldown ثانیه؛ حداکثر ۵ ارسال در پنجرهٔ ۱۵ دقیقه.
- محدودیت verify: حداکثر ۵ تلاش؛ سپس حذف challenge.

- TTL پیش‌فرض: ۵ دقیقه؛ index TTL روی `expiry` در MongoDB.
- `send-otp`: اگر کاربر با آن شماره وجود نداشته باشد، همان پاسخ موفق یکسان برمی‌گردد (بدون افشای ثبت‌نام).
- SMS: فقط با `SMS_ENABLED=true`؛ در production بدون SMS، OTP ارسال نمی‌شود (هشدار در لاگ).
- فرانت: `LoginView.tsx` — ورودی ۶ رقمی.

۲.۳ احراز هویت (`middleware/authController.js`) JWT

- Bearer token در `Authorization` header.
- `adminOnly` برای مسیرهای مدیریت.
- انقضای token قابل تنظیم با `JWT_EXPIRES_IN` (پیش‌فرض 7d).

۲.۴ زمین‌ها — ادمین (`routes/pitches.js, lib/pitchAdminFields.js`)

- POST/PATCH/DELETE زمین: `adminSecret` middleware.
- فیلدهای قابل نوشتن ادمین در آرایهٔ ثابت؛ سایر کلیدهای body نادیده گرفته می‌شوند.
- `commissionAmount` فقط توسط ادمین (در لیست سفید ادمین)، نه مالک.

۲.۵ مالک (`routes/owner.js`)

- `loadOwnedPitch`: فقط مالک همان زمین یا admin.
- PATCH زمین: whitelist فیلدها (قیمت، آدرس، last-minute و ... بدون کمیسیون).
- تغییر وضعیت رزرو به paid: در production مسدود مگر `ALLOW_OWNER_MANUAL_PAY=true`.

۲.۶ پرداخت (`routes/payment.js, services/paymentConfirm.js`)

- درخواست درگاه: JWT اجباری؛ رزرو با `user` لاگین‌شده؛ جمع مبلغ از رکوردهای DB.
- callback: verify با API زرین‌پال؛ سپس `confirmReservationPayment`.
- تأیید اتمی: `findOneAndUpdate` با شرط «هنوز successful نشده» و booking معتبر؛ جلوگیری از double-spend روی یک رزرو.
- کمیسیون از snapshot رزرو یا مقدار فعلی زمین در لحظهٔ تأیید.

۲.۷ پنالتی (`lib/penaltyReward.js, routes/discounts.js`)

- جریان: POST `/penalty-reward/session` → بازی → POST `/penalty-reward` با sessionId.
- sessionId: ۲۴ بایت random hex؛ یک‌بار مصرف با `usedAt`؛ انقضا ۲۰ دقیقه (قابل تنظیم).
- حداقل مدت بین ایجاد و claim در production: ۴۵ ثانیه (جلوگیری از اسکریپت فوری).
- rate limit: `RATE_LIMIT_PENALTY_MAX` (پیش‌فرض ۲۰ در ۱۵ دقیقه).

۲.۸ کیف پول و mock (`routes/wallet.js`, `routes/reservations.js`)

- topup و mock pay فقط development یا با flag صریح.
- سقف و کف مبلغ topup آزمایشی.
- تراکنش‌های wallet فقط برای کاربر لاگین‌شده؛ limit query برای لیست.

۲.۹ فرانت (`web/middleware.ts`)

- در production مسیرهای تست پرداخت و mobile-preview بسته می‌شوند.
- `/responsive-preview` برای بررسی UI مجاز است (با دادهٔ demo/preview).

۳. فایل‌ها و ماژول‌های مهم

مسیر	نقش امنیتی
`backend/server.js`	Helmet, CORS, rate limits, health, error handler
`backend/lib/rateLimits.js`	authLimiter و apiLimiter
`backend/lib/otpStore.js`	صدور و verify OTP
`backend/models/OtpChallenge.js`	ذخیرهٔ challenge در DB
`backend/lib/pitchAdminFields.js`	whitelist فیلد زمین (ادمین)
`backend/lib/penaltyReward.js`	نشست بازی پنالتی
`backend/models/PenaltyGameSession.js`	ذخیرهٔ session
`backend/lib/sheba.js`	اعتبارسنجی شبا
`backend/scripts/security-smoke-test.js`	۹ آزمون دود
`render.example.yaml`	نمونه env امن Render
`docs/deploy-security-checklist-fa.md`	چک‌لیست Atlas / Render / DNS
`web/middleware.ts`	بستن مسیرهای dev در production

۴. متغیرهای محیط (env) مرتبط با امنیت

جزئیات کامل در `backend/.env.example` و `render.example.yaml`.

متغیر	production (توصیه)
NODE_ENV	production
JWT_SECRET	تصادفی ≥ ۳۲ کاراکتر؛ فقط در پنل میزبانی
MONGO_URI	فقط سرور؛ IP allowlist در Atlas
CORS_ORIGINS	دامنه(های) واقعی فرانت
ALLOW_MOCK_PAY / ALLOW_MOCK_REFUND	false
ALLOW_OWNER_MANUAL_PAY	false (مگر سیاست کسب‌وکار)
ZARINPAL_SANDBOX	false + merchant واقعی
SMS_ENABLED	true + کلید کاوه‌نگار
OTP_PEPPER	رشتهٔ جدا از JWT (اختیاری ولی توصیه‌شده)
*_RATE_LIMIT	تنظیم بر اساس ترافیک

۵. آزمون‌های خودکار (Security Smoke Test)

اسکریپت: backend/scripts/security-smoke-test.js — اجرا: cd backend && npm run security:smoke

#	سناریو	انتظار
1	GET /api/health/live	200 و live: true
2	GET /api/reservations بدون token	401
3	POST /api/pitches بدون auth	401 یا 403
4	GET pitches با Origin evil.example	بدون ACAO برای مبدأ جعلی
5	send-otp برای شماره	200 یکسان (anti-enumeration)
6	verify-otp با کد غلط	400 یا 429
7	penalty-reward بدون sessionId	400
8	ایجاد session پنالتی	200 + sessionId
9	penalty-reward با session نامعتبر	400

نتیجهٔ مورد انتظار در محیط لوکال با بک‌اند روشن: **PASS ۹/۹**.

۶. چک‌لیست استقرار در پنل (خلاصه)

نسخهٔ کامل: `docs/deploy-security-checklist-fa.md`

MongoDB Atlas

- کاربر DB با حداقل دسترسی؛ TLS؛ Network Access محدود (نه `0.0.0.0/0` دائمی).
- Cloud Backup و تست restore فصلی روی staging.

Render (API)

- env طبق `render.example.yaml`؛ health check روی `/api/health/`.
- mock pay/refund و manual pay مالک غیرفعال.

فرانت / DNS

- هیچ secret در `NEXT_PUBLIC_*`؛ HTTPS؛ CDN/WAF (مثلاً Cloudflare) برای DDoS لایه شبکه.
- در صورت نشت secret در Git: rotate فوری (JWT، درگاه، SMS، پسورد DB).

۷. کارهای باقی‌مانده و محدودیت‌ها

> **خارج از کد — الزامی برای امنیت واقعی**
>
> Express و rate limit داخلی در برابر حملات بزرگ DDoS کافی نیستند؛ WAF/CDN لازم است. Atlas باید IP محدود شود. SMS باید در production فعال باشد وگرنه OTP عملاً کار نمی‌کند.

- **آزمون نشده در smoke:** رزرو هم‌زمان دو کاربر روی یک slot؛ callback واقعی زرین‌پال؛ بازیابی backup Atlas.
- **npm audit:** پس از hardening هنوز advisoryهای high ممکن است باقی بماند — npm audit و به‌روزرسانی دوره‌ای.
- **ADMIN_SECRET:** اختیاری برای اسکریپت‌ها؛ در production ترجیحاً فقط JWT admin.
- **فایل `render.yaml` حذف و به `render.example.yaml` جایگزین شد** تا secret در repo commit نشود.

۸. دستورات اجرا

```
cd backend
npm install
npm start

# در ترمینال دیگر:
cd backend
npm run security:smoke

# یا با آدرس staging:
node scripts/security-smoke-test.js https://your-api.example.com
```

— پایان گزارش —

برای به‌روزرسانی این PDF، فایل `docs/amniat-sanschii.html` را ویرایش و دوباره به PDF تبدیل کنید (چاپ از مرورگر → Save as PDF، یا ابزار headless Chrome).